

MÔN HỌC: THỊ GIÁC MÁY TÍNH (INF03111)

CHƯƠNG 2: NÂNG CẤP VÀ KHÔI PHỤC ẢNH

Tiết 3: Thực hành Lab 2.1 (Phần 2)
& Kiểm tra thường xuyên A1

Giảng viên: Trần Thành Thắng

Lớp: IT23M

Chủ đề: Tối ưu hóa xử lý ảnh phi tuyến tính



Lộ trình Tiết học: Từ Tuyển tính đến Phi tuyển tính

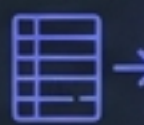
1. Ôn tập (Đã xong): Biến đổi tuyển tính (Brightness/Contrast)



2. Trọng tâm: Gamma Correction (Mô phỏng mắt người)



3. Kỹ thuật tối ưu: Look-Up Table (Tăng tốc 20x)



4. Nâng cao: CLAHE (Cân bằng Histogram thích ứng)



5. Đánh giá: Bài kiểm tra A1 (20 phút cuối)



Insight: Hôm nay chúng ta sẽ giải quyết bài toán: Làm sao để chỉnh sáng ảnh tự nhiên như mắt người mà không làm treo máy tính?

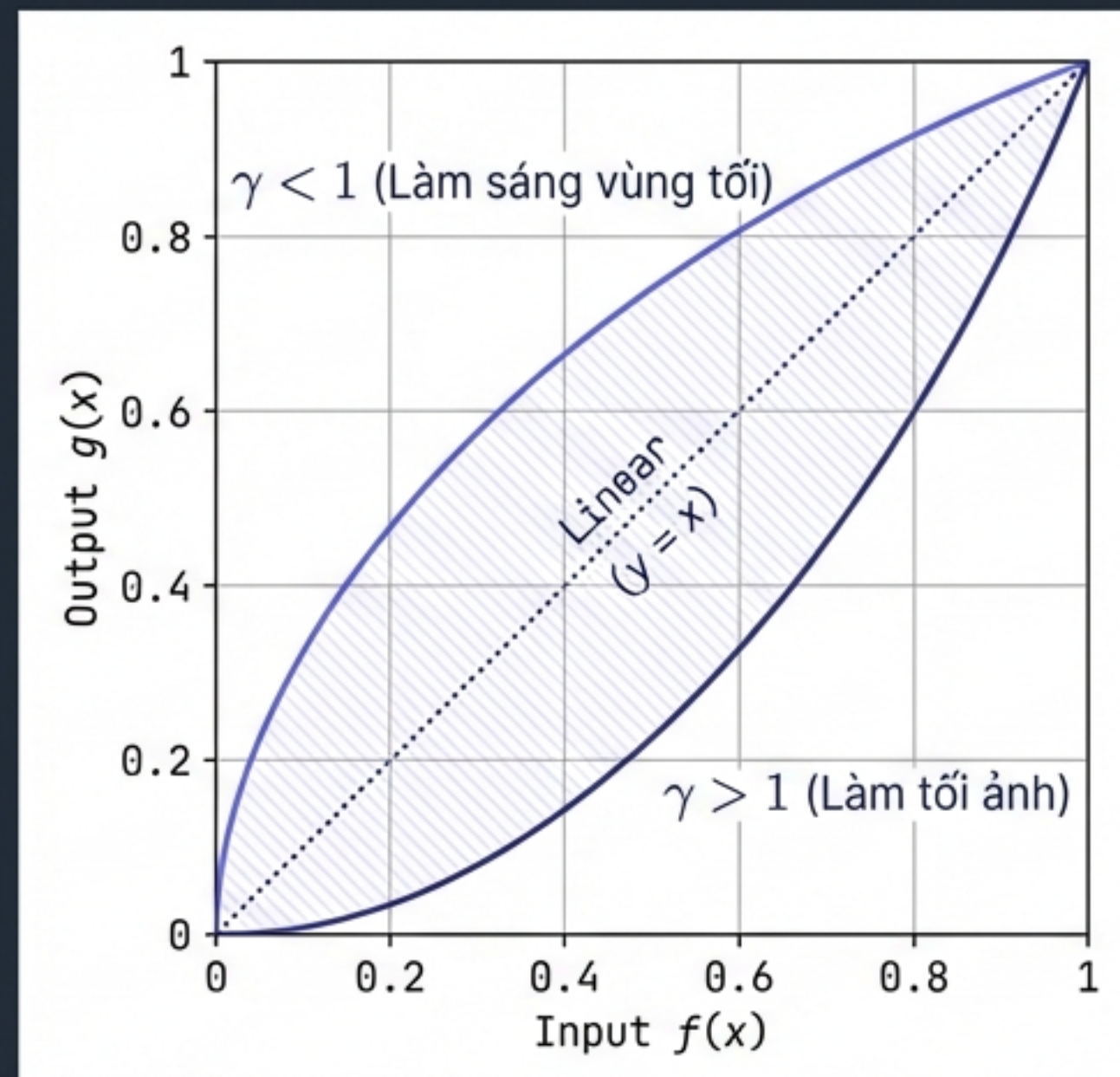
Tại sao tuyến tính là không đủ? (Gamma Correction)

Concept: Mắt người không cảm nhận ánh sáng theo đường thẳng. Chúng ta nhạy cảm hơn với sự thay đổi ở vùng tối (shadows) so với vùng sáng (highlights).

Công thức:

$$g(x) = c \cdot f(x)^\gamma$$

Trong đó: $f(x)$ là giá trị pixel đầu vào, γ là tham số điều chỉnh.



Kéo dẫn độ tương phản phi tuyến tính giúp khôi phục chi tiết ẩn trong bóng tối.

Vấn đề hiệu năng: Cái giá của phép toán dấu phẩy động

Cách làm ngây thơ (Naive approach): Chạy vòng lặp tính toán số thực cho từng điểm ảnh.

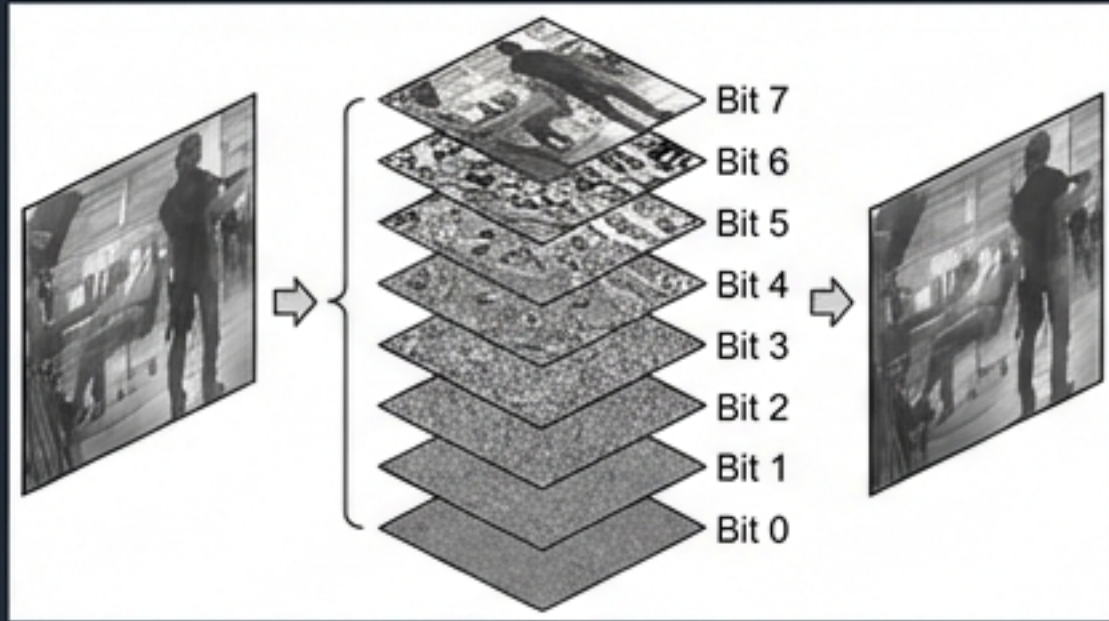


Input: 1920 x 1080 = 2,073,600 Pixels

Hậu quả: FPS thấp, lag, tốn tài nguyên.

Câu hỏi: Làm sao để áp dụng công thức phức tạp này cho 2 triệu điểm ảnh trong tích tắc?

Giải pháp tối ưu: Bảng tra cứu (Look-Up Table - LUT)



Ảnh 8-bit chỉ có giá trị từ 0 đến 255 (256 giá trị). Thay vì tính 2 triệu lần, ta chỉ tính trước 256 lần.

Pre-calculated Gamma Curve

0	0
1	17
2	37
3	40
...	60
...	100
100	150
10	140
10	120
...	...
250	200
255	255

Input Pixel Value
(e.g., "100")

Output Value
(150)

OpenCV: `output = cv2.LUT(input_image, table)`
Kết quả: Tốc độ xử lý tăng 20x-30x.

Thực hành Code: Cài đặt Gamma với LUT

```
# 1. Chuẩn bị bảng LUT (Look-Up Table)
```

```
# invGamma: Gamma nghịch đảo
```

```
invGamma = 1.0 / gamma
```

```
# Tạo mảng 256 giá trị đã tính trước
```

```
table = np.array([(i / 255.0) ** invGamma) * 255  
                  for i in np.arange(0, 256)]).astype("uint8")
```

```
# 2. Áp dụng bảng vào ảnh (Cực nhanh qua C++)
```

```
result = cv2.LUT(original_image, table)
```

Quan trọng: Phải đưa về kiểu số nguyên 8-bit.

Hàm được tối ưu hóa ở tầng thấp của OpenCV.

Kết quả trực quan: Tìm "Điểm ngọt" (Sweet Spot)

Original (Underexposed)



Gamma < 1.0 (Corrected)
(Sweet Spot)



Gamma > 1.0 (Darker)



Khuyến nghị: Sử dụng Trackbar (thanh trượt) trong OpenCV để thay đổi giá trị γ theo thời gian thực.

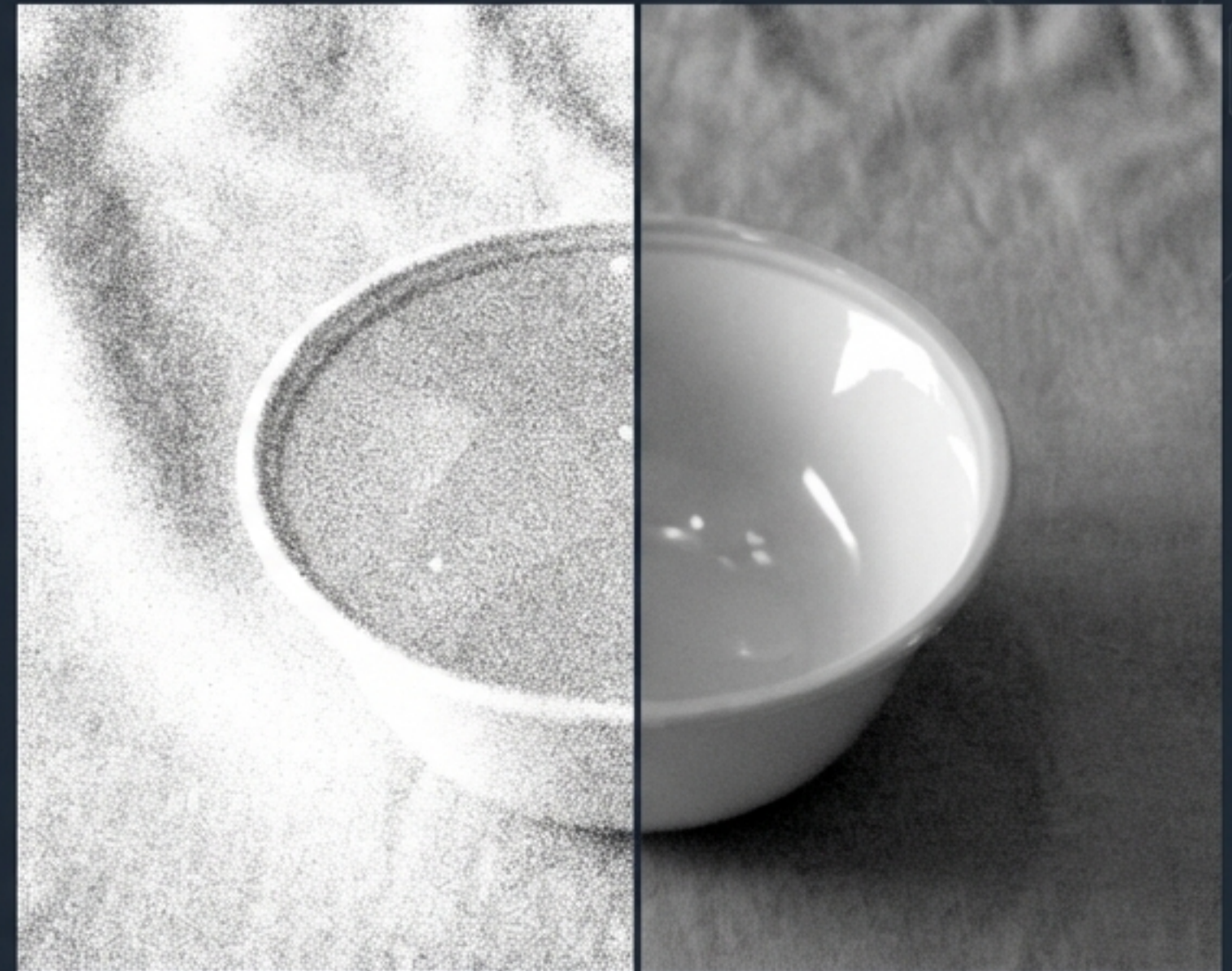
Từ Toàn cục đến Cục bộ: Giới thiệu CLAHE

Vấn đề của Global Equalization:

- Xử lý toàn bộ ảnh cùng lúc -> Vùng sáng bị cháy (washed out).
- Nhiễu (noise) bị khuếch đại ở vùng đồng nhất.

Giải pháp: CLAHE (Contrast Limited Adaptive Histogram Equalization)

- **Adaptive:** Chia ảnh thành lưới (grid).
- **Contrast Limited:** Giới hạn độ tương phản để chặn nhiễu.



Global EQ

CLAHE

Triển khai Code CLAHE

```
# Chuyển sang ảnh xám (Grayscale)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Khởi tạo đối tượng CLAHE
# clipLimit: Ngưỡng chặn nhiễu
# tileGridSize: Kích thước lưới cục bộ
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))

# Áp dụng
final_img = clahe.apply(gray)
```

Key Parameters

- **clipLimit=2.0**: "Giá trị an toàn. Tăng lên nếu ảnh quá mờ, giảm nếu ảnh quá nhiễu."
- **tileGridSize=(8,8)**: "Chia ảnh thành lưới 8x8 ô để xử lý cục bộ."

Tổng kết kỹ thuật: Khi nào dùng gì?

Kỹ thuật	Cơ chế	Ưu điểm	Nhược điểm	Ứng dụng
Linear	$y = ax + b$	Đơn giản, nhanh	Mất chi tiết vùng cực	Chỉnh sáng cơ bản
Gamma	Power Law + LUT	Mô phỏng mắt người	Cần tính bảng LUT	Ảnh thiếu sáng / ngược sáng
CLAHE	Adaptive Grid	Chi tiết cực bộ cực tốt	Có thể sinh nhiễu	Ảnh y tế, vệ tinh, sương mù

SẴN SÀNG CHO ĐÁNH GIÁ THƯỜNG XUYÊN A1

[ALERT BOX]

Thời gian: 20 phút

Hình thức: Live Coding (Google Colab/IDE)

Trọng số: 25% tổng điểm học phần

Hãy dọn dẹp màn hình, mở IDE và chuẩn bị file dữ liệu.



Đề bài A1: Cứu sáng ảnh đêm

Input Data: File
`night\_underexposed.jpg` (Tải từ Drive lớp học).

Nhiệm vụ: Bức ảnh chụp đêm bị mất chi tiết nghiêm trọng. Bạn cần khôi phục nó.

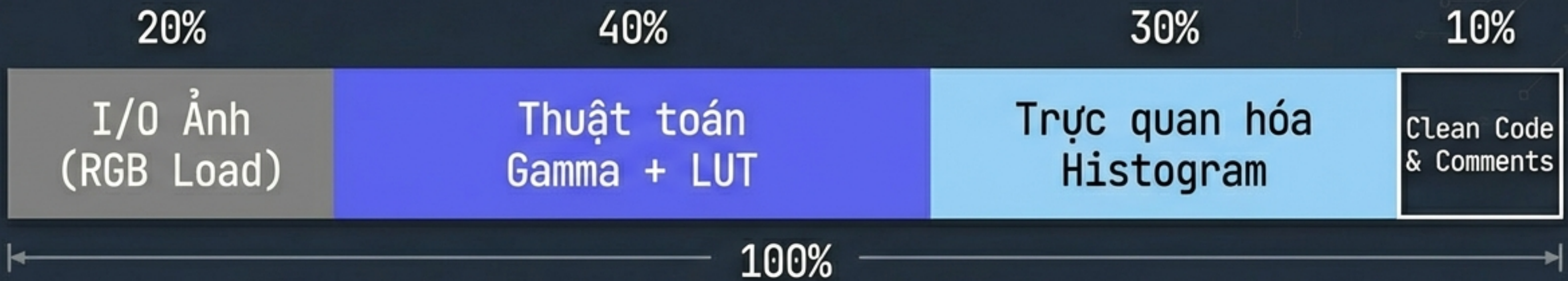


1. **Gamma Correction:** Làm sáng ảnh hợp lý, mắt người nhìn rõ chi tiết.
2. **Histogram Analysis:** Vẽ biểu đồ so sánh phân bố sáng trước và sau xử lý.

Yêu cầu Kỹ thuật (Bắt buộc)

- ❑ MUST USE: Phải sử dụng kỹ thuật **Look-Up Table** (LUT). Không dùng vòng lặp for duyệt từng pixel.
- ❑ VISUALIZATION: Hiển thị ảnh kết quả cạnh ảnh gốc. Chú ý chuyển đổi hệ màu BGR -> RGB.
- ❑ ANALYSIS: Vẽ 2 biểu đồ Histogram (Gốc vs. Đã xử lý) để chứng minh sự dịch chuyển dải sáng.

Tiêu chí chấm điểm (Rubric)



- ☐ 40% điểm nằm ở việc cài đặt đúng logic LUT và kiểu dữ liệu uint8.
- ☐ 30% điểm nằm ở khả năng phân tích biểu đồ (vẽ đúng trục, tiêu đề).

Tổng kết & Nhiệm vụ tuần tới

Tổng kết tuần này

- Hôm nay: Đã hoàn thành Xử lý Điểm ảnh (Point Processing).
- Bài tập về nhà: Nộp bài A1 lên LMS (.ipynb/.pdf).

Nội dung tuần tới (Xem trước)

- Tuần sau: Xử lý Vùng lân cận (Neighborhood Operations)
- Chủ đề: Tích chập (Convolution) & Lọc nhiễu.
- Từ khóa: Gaussian Blur, Sobel Edge Detection.

